

Fase 4

Dividir Para Programar: Modularização

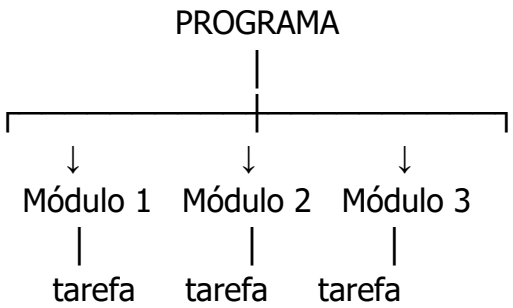
Try, Catch & Learn

Fase 4
Modularização: organizando o código em blocos reutilizáveis

Ao final da aula, o participante deverá entender o que é modularização, o porquê dividir um programa em partes facilita sua construção, diferenciar procedimentos e funções e reutilizar um mesmo bloco de código.

1. O que é modularização?

Modularização é dividir um programa grande em partes menores, chamadas de módulos.
Cada módulo fica responsável por uma tarefa específica.
Podemos imaginar um programa como um brinquedo de Lego 🧱:



Cada peça possui uma função dentro do conjunto.
Um programa de cadastro poderia ser dividido em:

```
PROGRAMA
│
├── cadastrarPessoa()
├── calcularIdade()
├── validarCPF()
├── mostrarDados()
└── sair()
```

Em vez de colocar tudo dentro de uma única parte do código, cada tarefa fica organizada em seu próprio módulo.

2. Por que modularizar?

A modularização traz várias vantagens.

📌 Organização

O código fica dividido em partes menores.

📌 Reutilização

Podemos utilizar o mesmo módulo várias vezes.

📌 Manutenção

Se precisarmos corrigir alguma coisa, sabemos onde procurar.

📌 Legibilidade

O código fica mais fácil de entender.

📌 Trabalho em equipe

Diferentes pessoas podem trabalhar em diferentes módulos.

3. Antes e depois da modularização

Imagine este programa:

```
#include <stdio.h>

int main() {

    printf("===== MENU =====\n");
    printf("1 - Cadastrar\n");
    printf("2 - Consultar\n");
    printf("3 - Sair\n");

    printf("\n===== MENU =====\n");
    printf("1 - Cadastrar\n");
    printf("2 - Consultar\n");
    printf("3 - Sair\n");

    return 0;
}
```

Perceba que o menu foi repetido.

Podemos transformar o menu em um módulo:

```
void mostrarMenu() {

    printf("===== MENU =====\n");
    printf("1 - Cadastrar\n");
    printf("2 - Consultar\n");
    printf("3 - Sair\n");
}
```

E depois utilizá-lo:

```
int main() {

    mostrarMenu();
    mostrarMenu();

    return 0;
}
```

Portanto, agora sempre que precisarmos do menu:
mostrarMenu()

4. Procedimentos

Um procedimento é um bloco de código criado para realizar uma tarefa específica. Em C, podemos representar um procedimento por meio de uma função com retorno `void`.

```
void mostrarMensagem() {
    printf("Olá, aluno!\n");
}
```

Esse código cria o procedimento.
Mas ele não será executado simplesmente porque foi criado.

Precisamos chamá-lo:

```
mostrarMensagem();
```

5. Criando e chamando um procedimento

Exemplo completo:

```
#include <stdio.h>

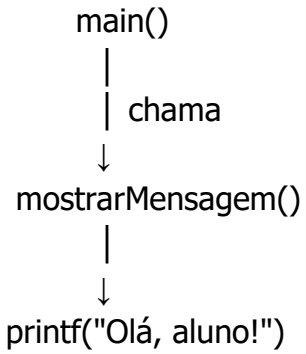
void mostrarMensagem() {
    printf("Olá, aluno!\n");
}

int main() {

    mostrarMensagem();

    return 0;
}
```

Fluxo:



6. Procedimentos podem ser reutilizados

Essa é uma das principais vantagens em se utilizar procedimentos.

```
int main() {

    mostrarMensagem();
    mostrarMensagem();
    mostrarMensagem();

    return 0;
}
```

Criamos o código uma vez e podemos utilizá-lo várias vezes.

7. Procedimentos com parâmetros

Depois que o conceito básico estiver claro, você pode apresentar parâmetros.
Um procedimento pode receber informações para realizar sua tarefa.

```
void mostrarDobro(int numero) {
    printf("Dobro: %d\n", numero * 2);
}
```

Chamadas:

```
mostrarDobro(5);
mostrarDobro(10);
```

Resultado:

```
Dobro: 10
Dobro: 20
```

int numero → é parâmetro
mostrarDobro(5); → passa o valor 5 para o procedimento

8. Funções

Assim como o procedimento, uma função é um bloco de código que realiza alguma tarefa, podendo receber parâmetros. Mas a diferença é que uma função tem valor de retorno, o que não acontece com procedimento.

```
int somar(int a, int b) {
    return a + b;
}
```

Uso:

```
int resultado = somar(5, 3);
```

Nesse caso:

somar(5, 3) → retorna o valor 8 → que é armazenado em resultado.